МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

Отчет к лабораторной работе № 1

Тема: **Порождающие паттерны: одиночка, строитель**

Дисциплина: **«Паттерны проектирования»**

Выполнила:

Студентка группы ПИЖ-б-о-23-2
направление подготовки 09.03.04
«Программная инженерия»

Силина Оксана Романовна

Проверил:

Ассистент ДЦРСИЭ

Белик А. И.

Ставрополь 2026

Цель работы: изучить принцип работы порождающего паттерна проектирования «Одиночка» и реализовать прикладную задачу расчёта траектории движения тела, брошенного под углом к горизонту, с использованием данного паттерна.

Задача: необходимо реализовать один из порождающих паттернов проектирования — «Строитель» или «Одиночка».

Индивидуальный вариант: 3. Разработка приложения «Траектория движения»

Репозиторий: https://github.com/tsukuri1/patterns

1. Описание паттерна «Одиночка»

Паттерн «Одиночка» (Singleton) — это порождающий паттерн проектирования, который гарантирует, что для определённого класса будет создан только один экземпляр, и предоставляет единую точку доступа к этому экземпляру.

Данный паттерн применяется в ситуациях, когда наличие нескольких экземпляров одного класса может быть нежелательным или не имеет смысла.

В разработанном приложении паттерн Singleton используется в классе TrajectoryCalculator. Данный класс отвечает за выполнение всех расчётов параметров движения тела.

Для реализации паттерна в классе используется переменная _instance и переопределённый метод __new__(). При создании объекта проверяется наличие уже существующего экземпляра класса. Если экземпляр отсутствует, он создаётся. При последующих обращениях возвращается уже существующий объект.

Таким образом, в приложении используется единый экземпляр TrajectoryCalculator.

2. Описание разработанного приложения

Разработанное приложение предназначено для расчёта параметров движения тела, брошенного под углом к горизонту.

Приложение реализовано на языке Python с использованием библиотеки Tkinter для создания графического интерфейса пользователя и библиотеки Matplotlib для построения графика траектории.

Основное окно приложения содержит поля для ввода начальной скорости и угла броска, кнопку «Рассчитать» и текстовое поле для вывода результатов.

К входным данным предъявляются следующие требования:

- начальная скорость должна быть больше 0;
- угол броска должен находиться в диапазоне от 0 до 90 градусов (не включая границы).

Приложение рассчитывает следующие параметры:

1. Время полёта — время, за которое тело вернётся на исходную высоту;
2. Максимальная высота подъёма — наибольшая высота, достигнутая телом;
3. Дальность полёта — расстояние по горизонтали от точки броска до точки падения.
4. Также реализована возможность построения графика траектории движения тела.

3. Диаграмма классов

Для отображения структуры разработанного приложения была создана UML-диаграмма классов.

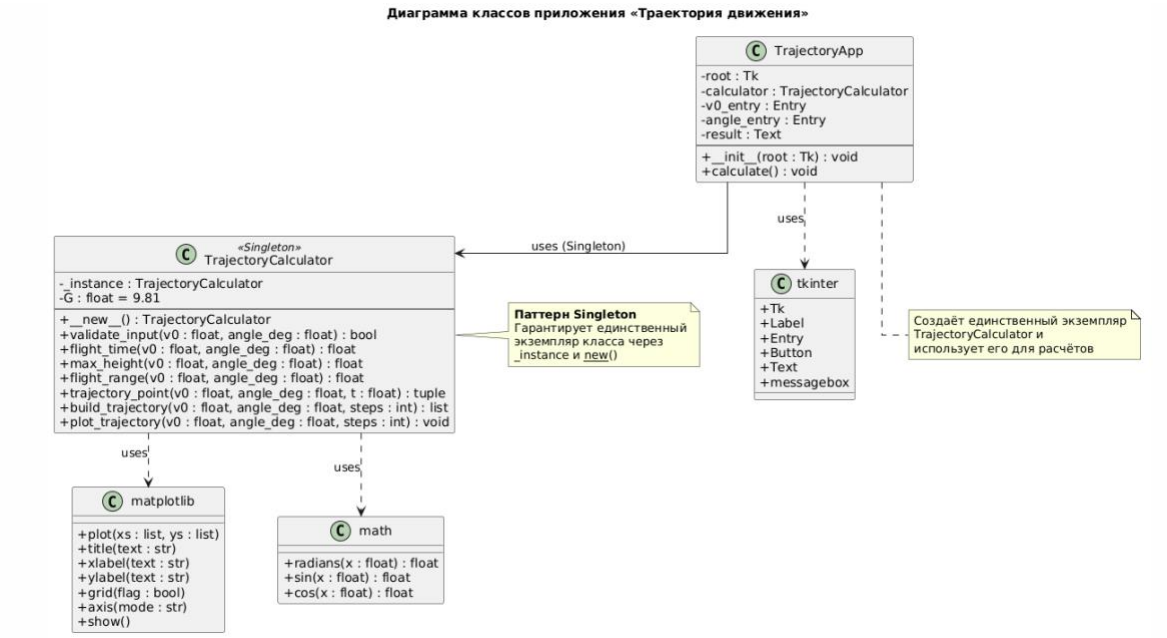


Рисунок 1 — Диаграмма классов приложения «Траектория движения»

Диаграмма содержит два основных класса: TrajectoryApp и TrajectoryCalculator.

Класс TrajectoryApp отвечает за создание графического интерфейса, обработку действий пользователя и взаимодействие с калькулятором траектории.

Класс TrajectoryCalculator реализует паттерн «Одиночка» и отвечает за выполнение всех расчётов.

Связь между классами показывает, что TrajectoryApp использует единственный экземпляр TrajectoryCalculator.

4. Реализация паттерна «Одиночка»

Основная часть реализации паттерна находится в классе TrajectoryCalculator.

```
python
class TrajectoryCalculator:
    _instance = None
```

```
def __new__(cls):
    if cls._instance is None:
        cls._instance = super().__new__(cls)
    return cls._instance
```

При первом создании объекта TrajectoryCalculator значение _instance отсутствует, поэтому создаётся новый объект.

При последующих попытках создать TrajectoryCalculator новый объект не создаётся. Метод __new__() возвращает уже существующий экземпляр.

Таким образом, класс TrajectoryCalculator реализует принцип Singleton.

5. Результаты работы программы

После запуска приложения пользователю отображается главное окно с полями ввода начальной скорости и угла броска.

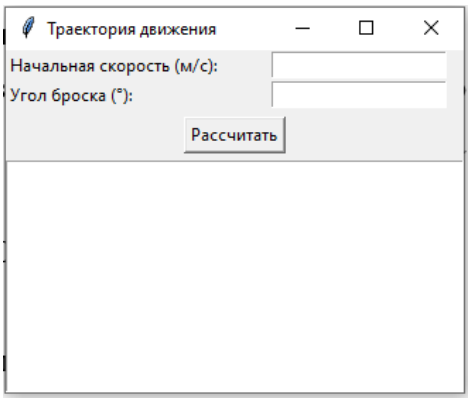


Рисунок 2 — Главное окно приложения

Для выполнения расчёта пользователь вводит значения начальной скорости и угла броска, после чего нажимает кнопку «Рассчитать».

Например:

- Начальная скорость: 20 м/с
- Угол броска: 45°

После нажатия кнопки программа выводит результаты расчёта и строит график траектории.

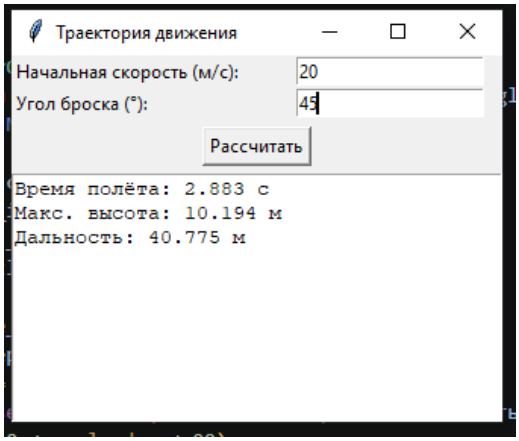


Рисунок 3 — Результаты расчёта

При правильном вводе программа выводит:

- Время полёта: 2.883 с
- Макс. высота: 10.194 м
- Дальность: 40.774 м

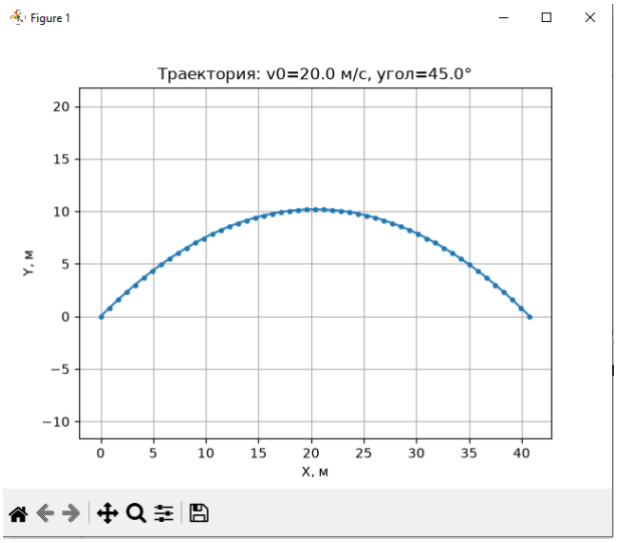


Рисунок 4 — График траектории движения

Если пользователь вводит некорректные данные (например, отрицательную скорость или угол вне допустимого диапазона), программа выводит сообщение об ошибке.

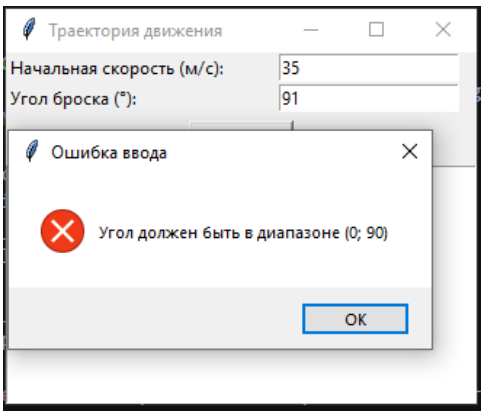


Рисунок 5 — Сообщение об ошибке ввода

6. Исходный код

Приложение разработано на языке программирования Python. Для создания графического интерфейса используется библиотека Tkinter, для построения графиков — библиотека Matplotlib.

```
import math
import tkinter as tk
from tkinter import messagebox
import matplotlib.pyplot as plt

G = 9.81


class TrajectoryCalculator:
    """Класс для расчёта траектории. Реализует паттерн Singleton."""
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

    def validate_input(self, v0, angle_deg):
        """Проверка корректности входных данных."""
        if v0 <= 0:
            raise ValueError("Начальная скорость должна быть > 0")
        if not (0 < angle_deg < 90):
            raise ValueError("Угол должен быть в диапазоне (0; 90)")
```

```
        return True

    def flight_time(self, v0, angle_deg):
        """Время полета тела."""
        self.validate_input(v0, angle_deg)
        angle_rad = math.radians(angle_deg)
        return 2 * v0 * math.sin(angle_rad) / G

    def max_height(self, v0, angle_deg):
        """Максимальная высота подъема."""
        self.validate_input(v0, angle_deg)
        angle_rad = math.radians(angle_deg)
        return (v0 ** 2) * (math.sin(angle_rad) ** 2) / (2 * G)

    def flight_range(self, v0, angle_deg):
        """Дальность полета."""
        self.validate_input(v0, angle_deg)
        angle_rad = math.radians(angle_deg)
        return (v0 ** 2) * math.sin(2 * angle_rad) / G

    def trajectory_point(self, v0, angle_deg, t):
        """Координаты точки траектории в момент времени t."""
        self.validate_input(v0, angle_deg)
        if t < 0 or t > self.flight_time(v0, angle_deg):
            raise ValueError("Время вне интервала полета")
        angle_rad = math.radians(angle_deg)
        x = v0 * math.cos(angle_rad) * t
        y = v0 * math.sin(angle_rad) * t - 0.5 * G * t ** 2
        return round(x, 3), round(y, 3)

    def build_trajectory(self, v0, angle_deg, steps=20):
        """Список точек траектории."""
        T = self.flight_time(v0, angle_deg)
        return [self.trajectory_point(v0, angle_deg, T * i / steps)
                for i in range(steps + 1)]

    def plot_trajectory(self, v0, angle_deg, steps=50):
        """Построение графика траектории."""
        pts = self.build_trajectory(v0, angle_deg, steps)
```

```
        xs = [p[0] for p in pts]
        ys = [p[1] for p in pts]
        plt.plot(xs, ys, marker='o', markersize=3)
        plt.title(f"Траектория: v0={v0} м/с, угол={angle_deg}°")
        plt.xlabel("X, м")
        plt.ylabel("Y, м")
        plt.grid(True)
        plt.axis('equal')
        plt.show()


class TrajectoryApp:
    """Класс графического интерфейса."""
    def __init__(self, root):
        self.root = root
        self.calculator = TrajectoryCalculator()
        root.title("Траектория движения")
        tk.Label(root, text="Начальная скорость (м/с):").grid(
            row=0, column=0, sticky="w")
        self.v0_entry = tk.Entry(root)
        self.v0_entry.grid(row=0, column=1)
        tk.Label(root, text="Угол броска (°):").grid(
            row=1, column=0, sticky="w")
        self.angle_entry = tk.Entry(root)
        self.angle_entry.grid(row=1, column=1)
        tk.Button(root, text="Рассчитать",
                  command=self.calculate).grid(
            row=2, column=0, columnspan=2, pady=5)
        self.result = tk.Text(root, width=40, height=10)
        self.result.grid(row=3, column=0, columnspan=2)

    def calculate(self):
        try:
            v0 = float(self.v0_entry.get())
            angle = float(self.angle_entry.get())
            self.calculator.validate_input(v0, angle)
            T = self.calculator.flight_time(v0, angle)
            H = self.calculator.max_height(v0, angle)
            L = self.calculator.flight_range(v0, angle)
```

```
            self.result.delete("1.0", tk.END)
            self.result.insert(tk.END,
                f"Время полёта: {T:.3f} с\n"
                f"Макс. высота: {H:.3f} м\n"
                f"Дальность: {L:.3f} м\n")
            self.calculator.plot_trajectory(v0, angle)
        except ValueError as e:
            messagebox.showerror("Ошибка ввода", str(e))
        except Exception:
            messagebox.showerror("Ошибка", "Некорректные данные")


if __name__ == "__main__":
    root = tk.Tk()
    app = TrajectoryApp(root)
    root.mainloop()
```

Вывод:

В ходе выполнения лабораторной работы было разработано приложение «Траектория движения» на языке программирования Python.

В приложении реализованы расчёт времени полёта, максимальной высоты подъёма, дальности полёта и построение графика траектории движения тела, брошенного под углом к горизонту.

В процессе разработки был изучен и реализован порождающий паттерн проектирования «Одиночка». Паттерн применён к классу TrajectoryCalculator, благодаря чему в приложении используется единственный экземпляр калькулятора траектории.

Таким образом, была изучена практическая реализация паттерна Singleton и получено работоспособное приложение с графическим интерфейсом для расчёта и визуализации траектории движения тела.